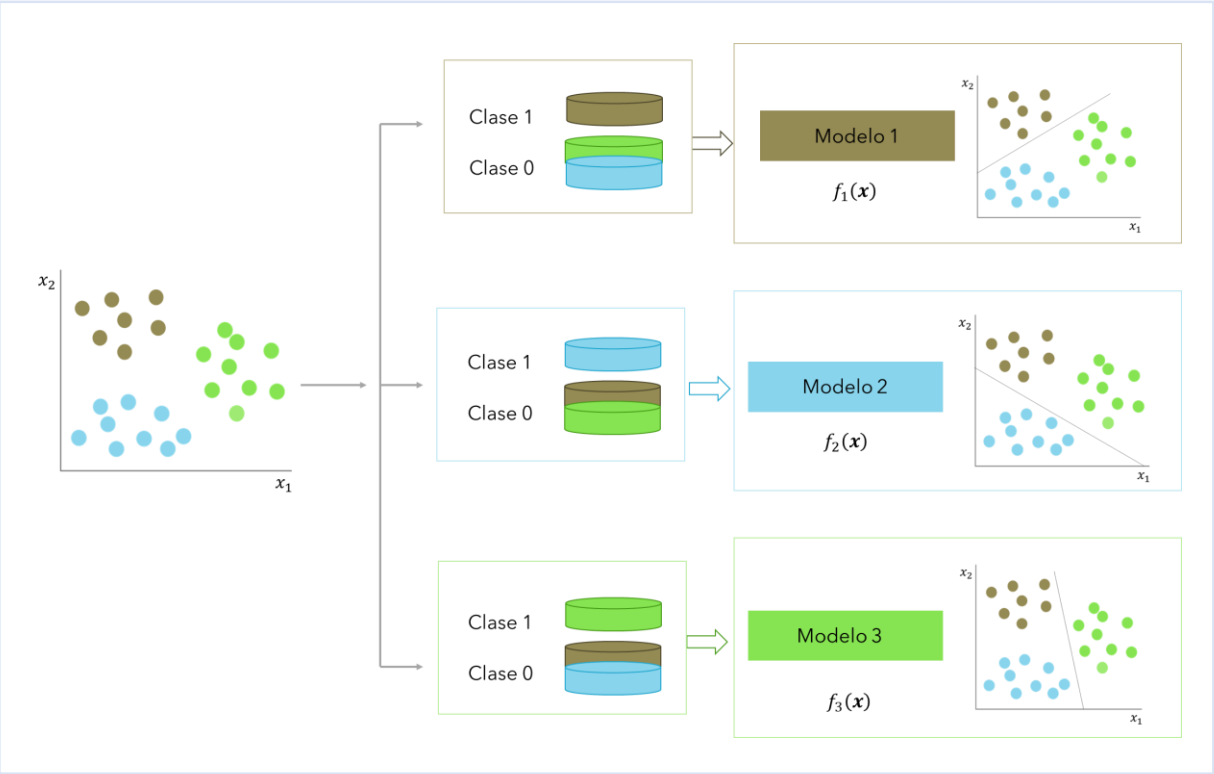
Sesión 5. Árboles de decisión. Combinación de modelos (*ensembles*)

- Curva precisión – recall y curva ROC
- Selección de características.
- Algoritmos basados en árboles de decisión
- Combinación de modelos (ensembles).

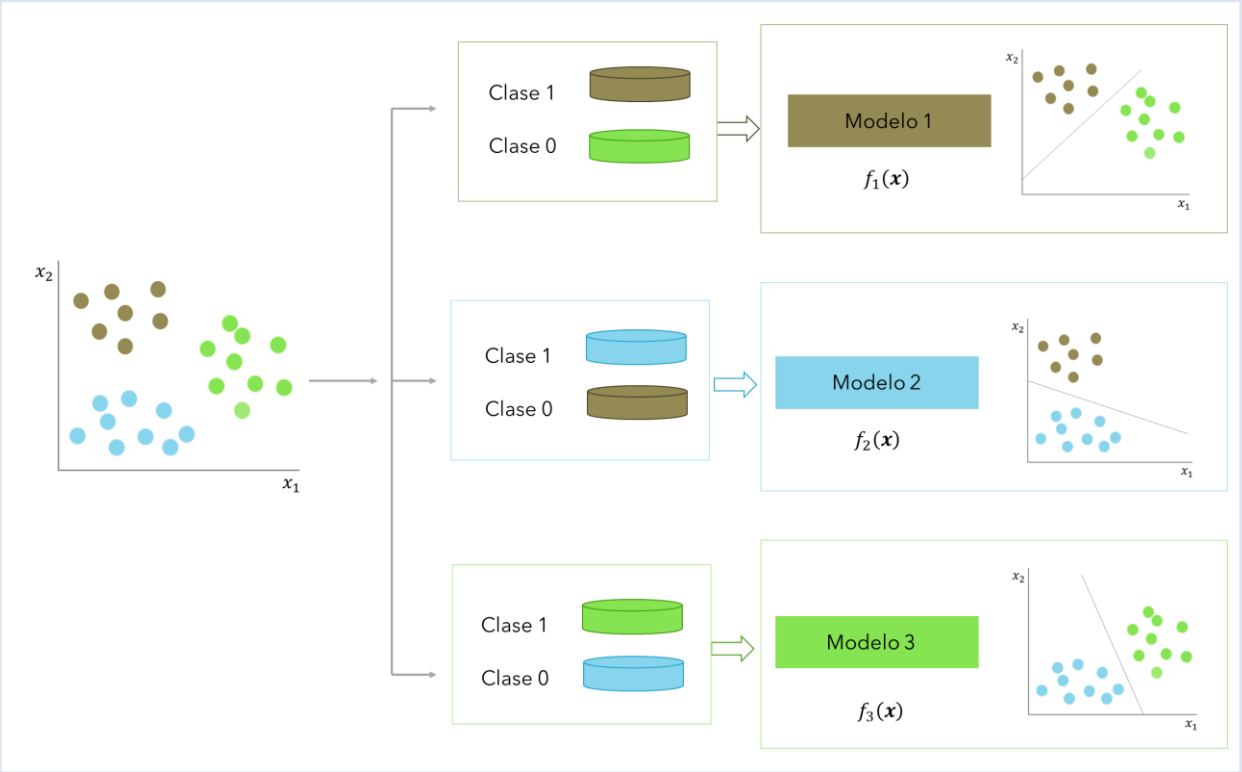
❑ Clasificadores binarios en contextos multiclase

OVA: uno-vs-el resto (one-vs-all)



Se obtiene la clasificación tomando el valor máximo entre todos los valores generados (probabilidad de la clase 1, para cada modelo).

OVO: uno-vs-uno (one-vs-one)



Se obtiene la clasificación tomando la clase mayoritaria (más votada).

❑ Además, para regresión logística:

- El modelo de regresión logística puede generalizarse para admitir múltiples clases directamente. Este modelo se conoce como **regresión logística multinomial** o regresión softmax. ¿Cómo funciona?

- Se aprende un grupo de coeficientes por cada clase $k \Rightarrow\ o_k = w_{k0} + w_{k1}x_1 + \ldots + w_{kd}x_d$

- Para clasificar una instancia $\boldsymbol{x}$, el modelo primero genera una puntuación $o(\boldsymbol{x})$ para cada clase k:

$$o(\boldsymbol{x}) = (o_1, o_2, \ldots, o_k) \quad donde\ o_i\ es\ la\ puntuación\ (score)\ de\ la\ clase\ i\ (i = 1..k)$$

y luego estima la probabilidad de cada una de estas aplicando la función softmax (también llamada función normalizada exponencial) a las puntuaciones. Las puntuaciones se denominan generalmente logits o log-odds.

$$softmax(o_i) = \frac{e^{o_i}}{\sum_{i=1}^{k} e^{o_i}}$$

- Al igual que el clasificador de regresión logística, el clasificador de regresión softmax predice la clase con la mayor probabilidad estimada (que es simplemente la clase con la puntuación más alta). Un aspecto para resaltar es que para entrenar este modelo no se utiliza la función de costo logarítmica, sino la función **entropía cruzada**, la cual es una generalización de la primera para problemas multiclase.

$$J(\boldsymbol{W}) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{k} y_i^k log(\hat{y}_i^k) \quad donde\ k = número\ de\ clases$$

Curva precisión – recall y curva ROC

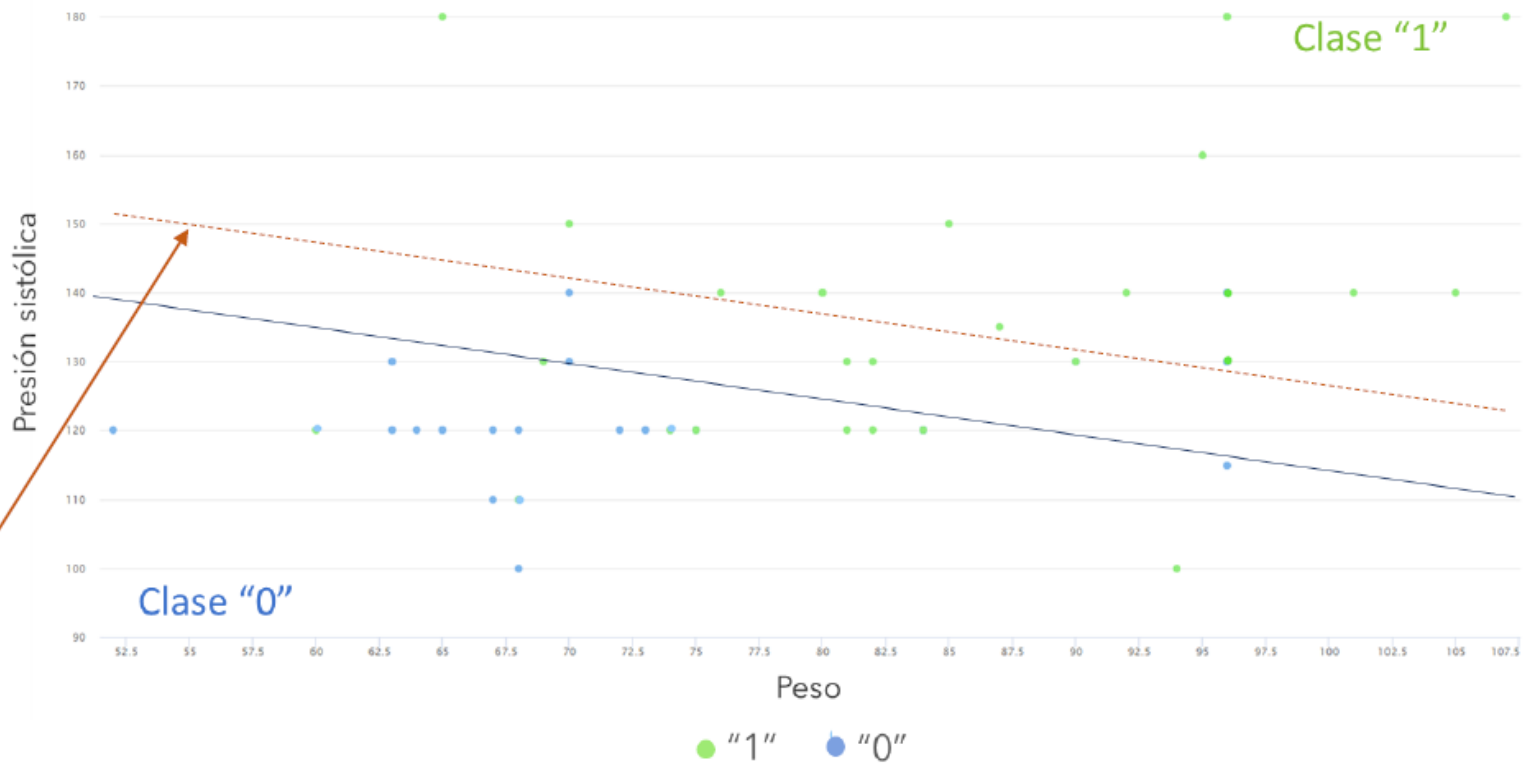
- Actualmente nuestro clasificador realiza las siguientes predicciones:

$$h = 0.017 \times peso + 0.066 \times presión\ sistólica - 9.15 = 0$$

$$clase\ 1: \quad f_{logística}(h) \geq 0.5$$
$$clase\ 0: \quad f_{logística}(h) < 0.5$$

- Sin embargo, pudiésemos estar interesados en generar una clasificación para un paciente solo cuando sea tenga una alta confianza en la predicción, es decir, con una mayor probabilidad. Por ejemplo, cuando esta sea de un 75%. En este caso:

$$clase\ 1 \text{ si} \quad f_{logística}(h) \geq 0.75$$
$$clase\ 0 \text{ si} \quad f_{logística}(h) < 0.75$$

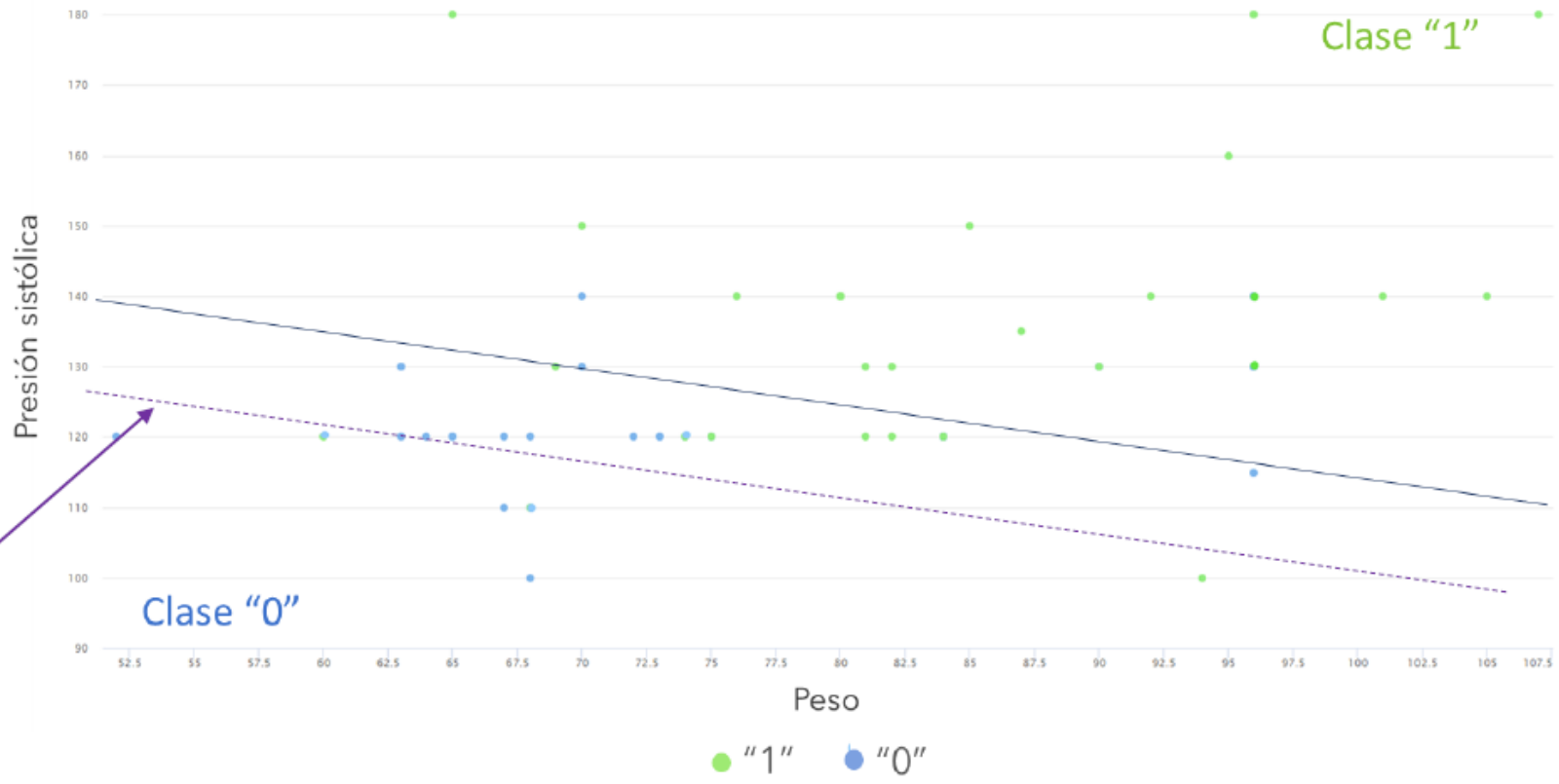


Se obtendría entonces un clasificador enfocado en la precisión:

Alta precisión, bajo recall

- En otro contexto, el interés podría ser generar una clasificación correcta para todos los pacientes que presentan esta enfermedad. En este caso, el umbral tendría que ser menor para garantizar que no quede ningún caso por fuera de la clasificación, por ejemplo:

$$\begin{array}{lll} clase\ 1 \text{ si} & f_{logística}(h) & \geq 0.25 \\ clase\ 0 \text{ si} & f_{logística}(h) & < 0.25 \end{array}$$

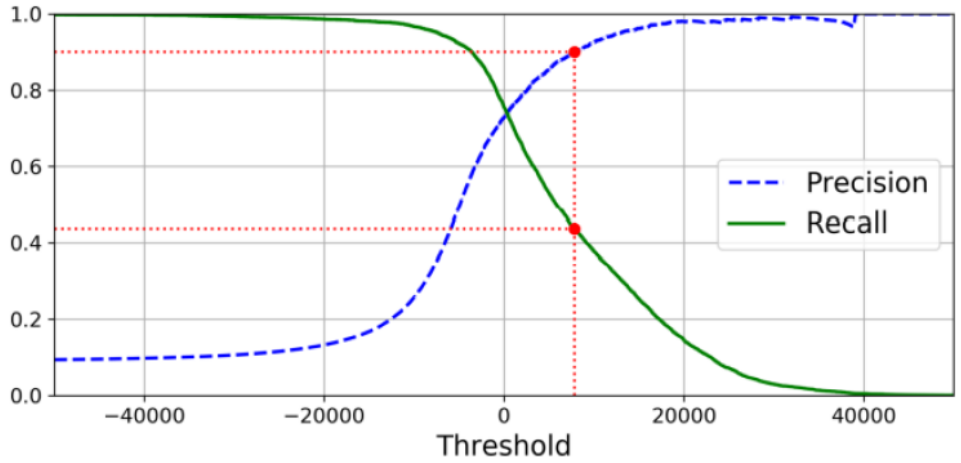


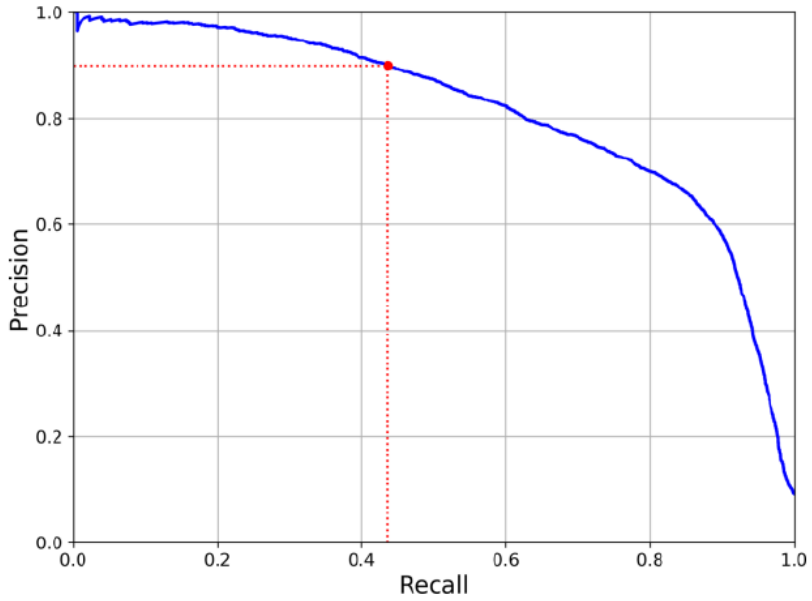
Se obtendría entonces un clasificador enfocado en la recall:

Alto recall, baja precisión

En algunas situaciones se presenta este compromiso entre el recall (sensibilidad) y la precisión. Este comportamiento puede ser visualizado mediante las curvas **precisión-recall**.

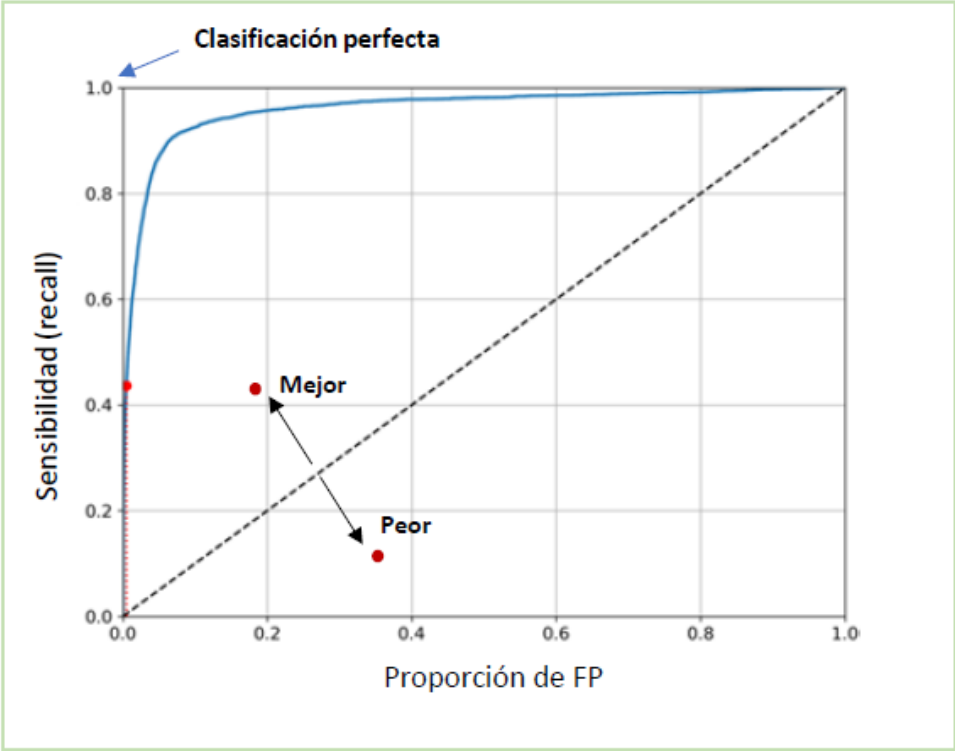
Curvas precisión - recall





Curva ROC

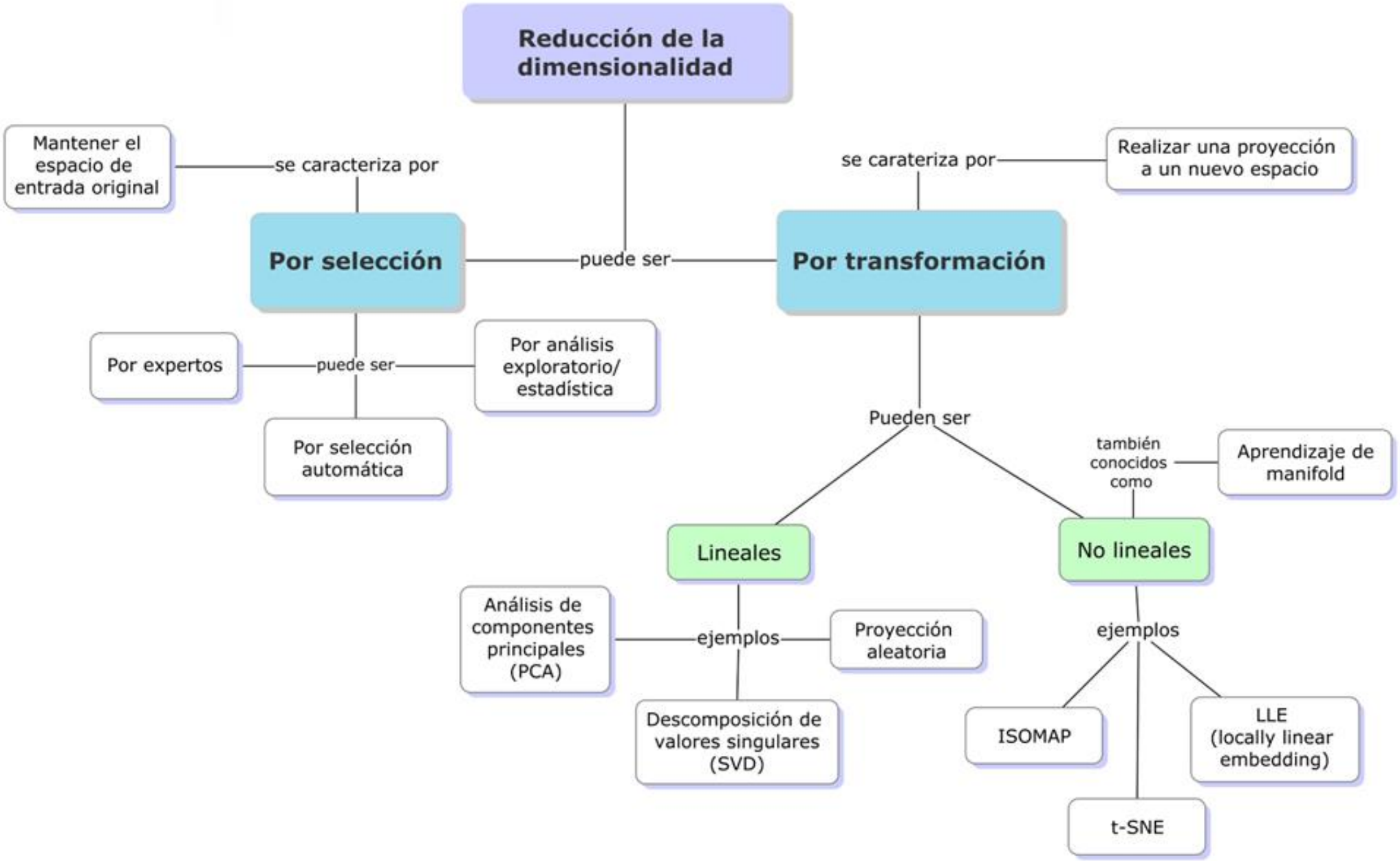
La curva ROC (*Receiver Operating Characteristic*) muestra gráficamente la relación entre la sensibilidad y la proporción de falsos positivos.



Se puede determinar el área total bajo la curva ROC (AUC), como una medida para evaluar el rendimiento de clasificadores

$$AUC = \frac{1 + \left(\frac{VP}{VP + FN}\right) - \left(\frac{FP}{VN + FP}\right)}{2}$$

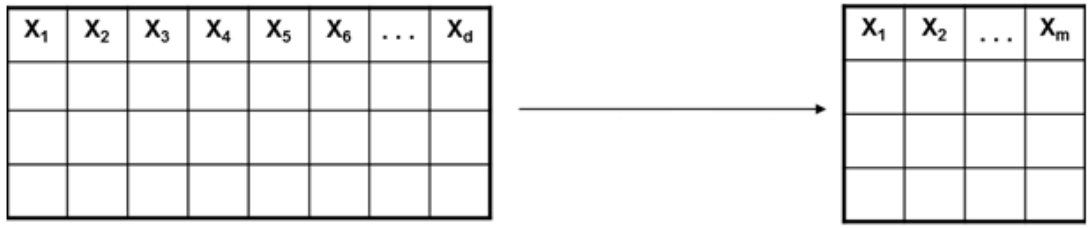
Selección de características



❑ Reducción por selección

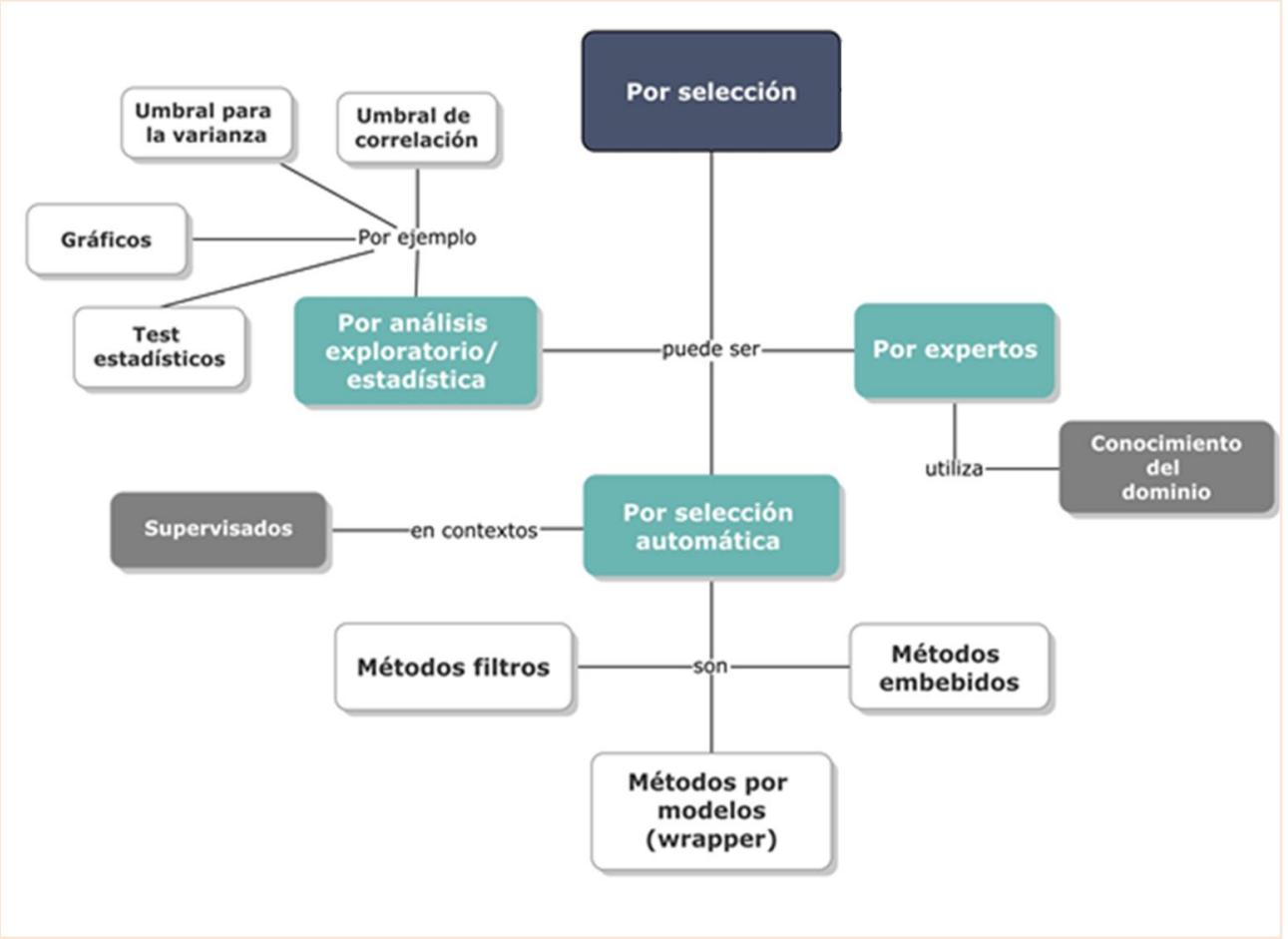
Seleccionar, a partir de d variables originales, un subconjunto de m características ($m \quad d$) que, idealmente, es el necesario para explicar el problema bajo estudio (variables más informativas).

Con la selección de variables se espera que $m \ll d$ con máxima información



No se cambia a un nuevo espacio de atributos ⇒ los atributos que conforman la nueva representación forman parte del espacio original.

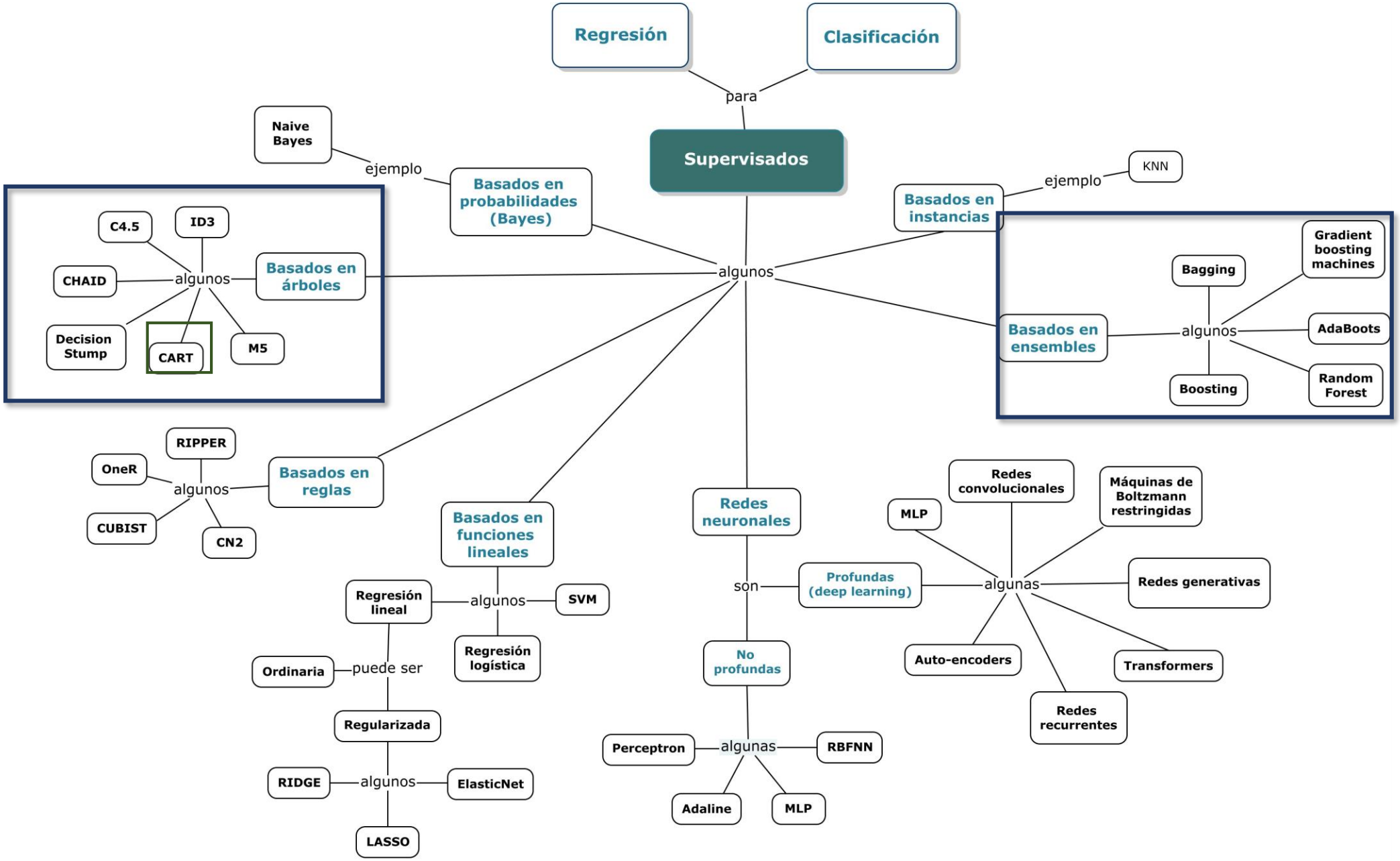
¿Cómo seleccionar?



Algoritmos basados en árboles de decisión

Antes: resumen de los algoritmos basados en funciones lineales:

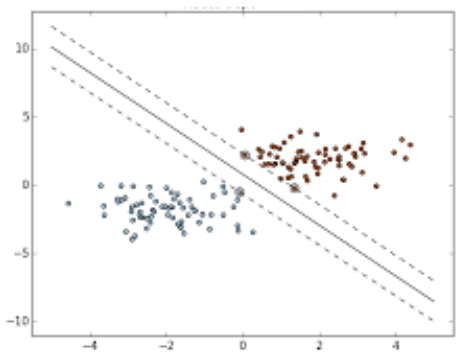
Algoritmo	Tarea	Parámetros del modelo	Control de la complejidad	Características y contextos de aplicación
Regresión lineal	Regresión	Coeficientes de la combinación lineal $\boldsymbol{W} = (w_1, \dots, w_d), b$	*Regresión ordinaria: No* Regresión Ridge: regularización L2. Regresión Lasso: regularización L1. Parámetro de regularización (hiperparámetro) = α. A mayor valor de α , más regularización (se obtienen modelos más simples)	Solo variables numéricas. Mejores resultados cuando se normalizan o estandarizan los datos. Modelos simples, fáciles de entrenar. Rapidez en las predicciones. Escalan bien a grandes conjuntos de datos. Trabajan bien con matrices de datos dispersas. Modelos interpretables.
Regresión logística	Clasificación	Coeficientes de la combinación lineal $\boldsymbol{W} = (w_1, \dots, w_d), b$	En general, regularización L2 (pero se pueden utilizar otros términos, como L1). Parámetro de regularización (hiperparámetro) = C A menor valor de C, más regularización (se obtienen modelos más simples).	
Máquinas de vectores de soporte	Clasificación y regresión	Al resolver el problema dual la función de decisión queda expresada por: los vectores de soporte, los α_i y la función Kernel.	Parámetro de regularización C y parámetros de la función Kernel. A menor valor de C, más regularización (se obtienen modelos más simples).	Solo variables numéricas. Importante la normalización o estandarización de las variables. Requiere de un ajuste cuidadoso de los hiperparámetros. Resultan adecuados para datos con alta dimensionalidad (aun con pocos datos). Son capaces de aprender límites de decisión muy complejos (con kernel). No escala bien con el número de datos. En estos contextos son computacionalmente costosos. Para contextos no balanceados se puede utilizar la SVM basada en costo. Para la detección de anomalías One-class SVM, la cual aprende un límite de decisión solo para la clase normal.


Regresión
Clasificación
para
Supervisados
Naive Bayes
ejemplo
Basados en probabilidades (Bayes)
algunos
Basados en instancias
ejemplo
KNN
C4.5
ID3
CHAID
algunos
Basados en árboles
Decision Stump
CART
M5
Gradient boosting machines
Bagging
Basados en ensembles
algunos
AdaBoots
Boosting
Random Forest
RIPPER
OneR
algunos
Basados en reglas
CUBIST
CN2
Basados en funciones lineales
Regresión lineal
algunos
SVM
Regresión logística
Ordinaria
puede ser
Regularizada
RIDGE
algunos
ElasticNet
LASSO
Redes neuronales
son
Profundas (deep learning)
No profundas
Perceptron
algunas
RBFNN
Adaline
MLP
Redes convolucionales
Máquinas de Boltzmann restringidas
MLP
algunas
Redes generativas
Auto-encoders
Transformers
Redes recurrentes

Algoritmo de aprendizaje

Mecanismo de aprendizaje

Lenguaje de representación



Basados en funciones lineales

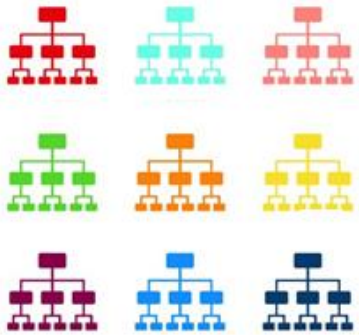
$$P(C_1|X) = \frac{P(X|C_1)P(C_1)}{P(X)}$$

$$P(C_2|X) = \frac{P(X|C_2)P(C_2)}{P(X)}$$

$$\vdots$$

$$P(C_k|X) = \frac{P(X|C_k)P(C_k)}{P(X)}$$

Basados en probabilidades



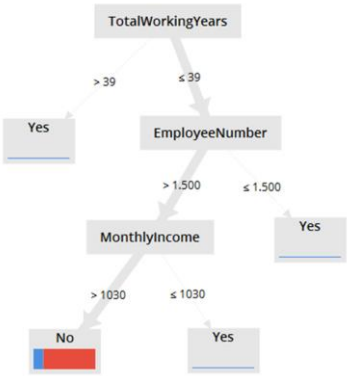
Basadas en combinación de modelos

Máquinas de vectores de soporte
Regresión logística, …

Naïve Bayes, …

Random Forest
Gradient Boosting …

C4.5
CART,…



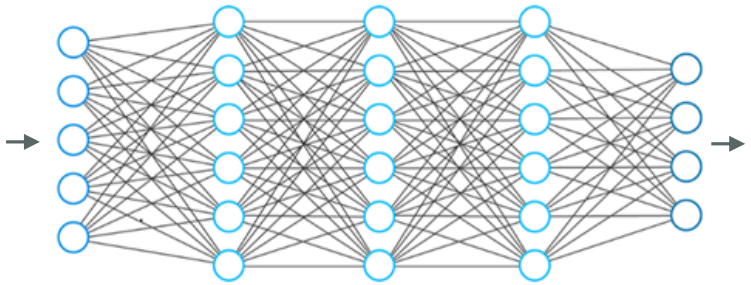
Basados en árboles

RIPPER,…

```
If Astigmatismo = No
  and ProducciónLágrimas = Normal
  and Prescipción = Hipermetropia then Blandos
If Astigmatismo = No
  and ProducciónLágrimas = normal
  and Edad = Joven then Blandos
If Edad = Pre-presbicia
  and Astigmatismo = No
  and ProducciónLágrimas = Normal then Blandos
If Astigmatismo = Si
  and ProducciónLágrimas = Normal
  and Prescipción = Miopia then Duros
If Edad = Joven
  and Astigmatismo = Si
  and ProducciónLágrimas = Normal then Duros
If ProducciónLágrimas = Reducida then Ninguno
If Edad = Presbicia
  and ProducciónLágrimas = Normal
  and Prescipción = Miopia
  and Astigmatismo = No then Ninguno
If Prescipción = Hipermetropía
  and Astigmatismo = Si
  and Edad = Pre-Presbicia then Ninguno
If Edad = Presbicia
  and Prescipción = Hipermetropía
  and Astigmatismo = Si then Ninguno
```

Basados en reglas

Perceptron multicapa (MLP),…

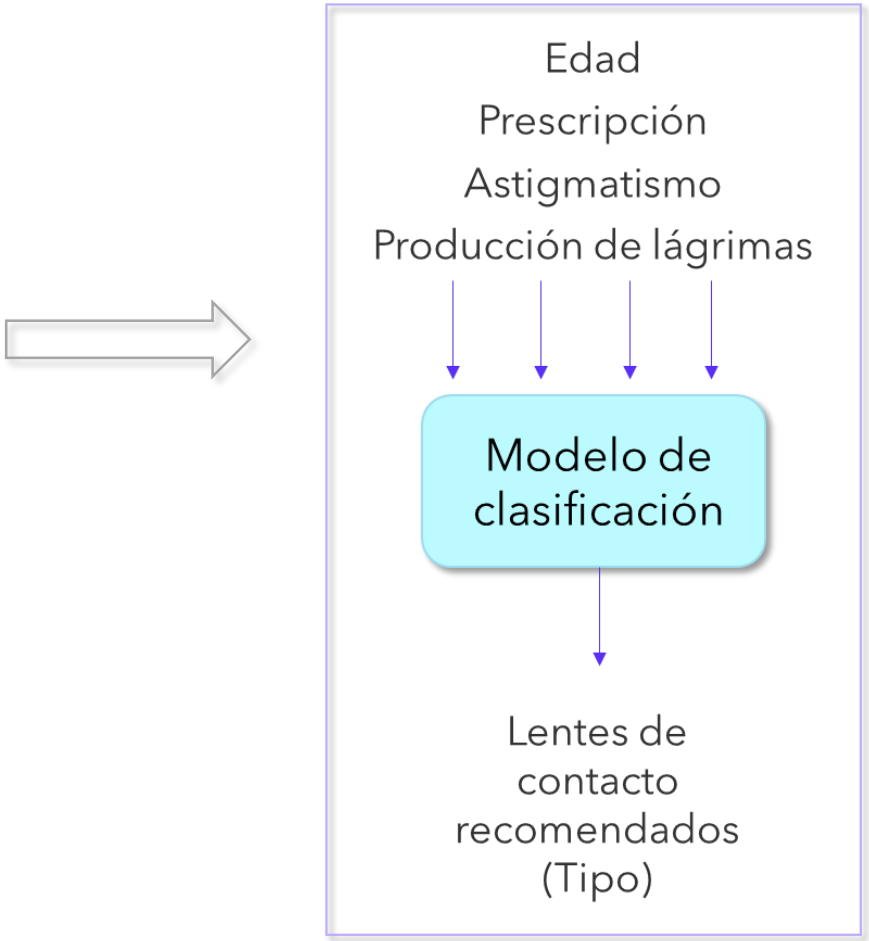


Redes neuronales

¿Cómo funcionan?

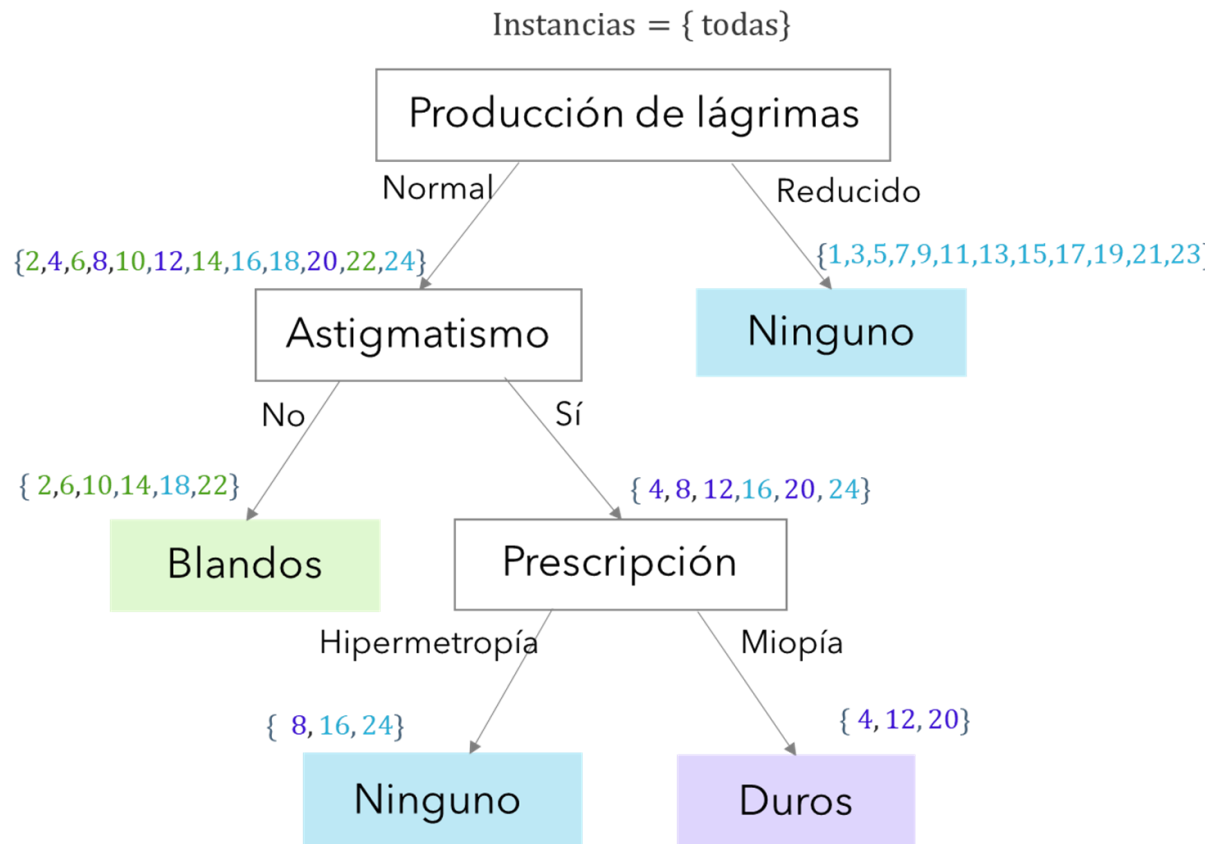
Se quiere construir un modelo de clasificación que realice una recomendación del tipo de lentes de contacto que debería usar un paciente con base en las variables edad, prescripción (miopía o hipermetropía), si tiene o no astigmatismo y su nivel de producción de lágrimas. La variable objetivo es el tipo de lentes de contacto recomendado. Se tienen tres categorías para este atributo clase: ninguno, duros y blandos.:

Instancia	Edad	Prescripción	Astigmatismo	Producción de lágrimas	Tipo
1	Joven	Miopía	No	Reducido	Ninguno
2	Joven	Miopía	No	Normal	Blandos
3	Joven	Miopía	Sí	Reducido	Ninguno
4	Joven	Miopía	Sí	Normal	Duros
5	Joven	Hipermetropía	No	Reducido	Ninguno
6	Joven	Hipermetropía	No	Normal	Blandos
7	Joven	Hipermetropía	Sí	Reducido	Ninguno
8	Joven	Hipermetropía	Sí	Normal	Duros
9	Pre-présbita	Miopía	No	Reducido	Ninguno
10	Pre-présbita	Miopía	No	Normal	Blandos
11	Pre-présbita	Miopía	Sí	Reducido	Ninguno
12	Pre-présbita	Miopía	Sí	Normal	Duros
13	Pre-présbita	Hipermetropía	No	Reducido	Ninguno
14	Pre-présbita	Hipermetropía	No	Normal	Blandos
15	Pre-présbita	Hipermetropía	Sí	Reducido	Ninguno
16	Pre-présbita	Hipermetropía	Sí	Normal	Ninguno
17	Présbita	Miopía	No	Reducido	Ninguno
18	Présbita	Miopía	No	Normal	Ninguno
19	Présbita	Miopía	Sí	Reducido	Ninguno
20	Présbita	Miopía	Sí	Normal	Duros
21	Présbita	Hipermetropía	No	Reducido	Ninguno
22	Présbita	Hipermetropía	No	Normal	Blandos
23	Présbita	Hipermetropía	Sí	Reducido	Ninguno
24	Présbita	Hipermetropía	Sí	Normal	Ninguno



Instancia	Edad	Prescripción	Astigmatismo	Producción de lágrimas	Tipo
1	Joven	Miopía	No	Reducido	Ninguno
2	Joven	Miopía	No	Normal	Blandos
3	Joven	Miopía	Sí	Reducido	Ninguno
4	Joven	Miopía	Sí	Normal	Duros
5	Joven	Hipermetropía	No	Reducido	Ninguno
6	Joven	Hipermetropía	No	Normal	Blandos
7	Joven	Hipermetropía	Sí	Reducido	Ninguno
8	Joven	Hipermetropía	Sí	Normal	Duros
9	Pre-présbita	Miopía	No	Reducido	Ninguno
10	Pre-présbita	Miopía	No	Normal	Blandos
11	Pre-présbita	Miopía	Sí	Reducido	Ninguno
12	Pre-présbita	Miopía	Sí	Normal	Duros
13	Pre-présbita	Hipermetropía	No	Reducido	Ninguno
14	Pre-présbita	Hipermetropía	No	Normal	Blandos
15	Pre-présbita	Hipermetropía	Sí	Reducido	Ninguno
16	Pre-présbita	Hipermetropía	Sí	Normal	Ninguno
17	Présbita	Miopía	No	Reducido	Ninguno
18	Présbita	Miopía	No	Normal	Ninguno
19	Présbita	Miopía	Sí	Reducido	Ninguno
20	Présbita	Miopía	Sí	Normal	Duros
21	Présbita	Hipermetropía	No	Reducido	Ninguno
22	Présbita	Hipermetropía	No	Normal	Blandos
23	Présbita	Hipermetropía	Sí	Reducido	Ninguno
24	Présbita	Hipermetropía	Sí	Normal	Ninguno

Algoritmo basado en árboles de decisión



Estrategia para la inducción de árboles de decisión:

☞ Dividir los registros (espacio de entrada), utilizando un test de atributo que optimiza un cierto criterio.

Por ejemplo, un criterio a optimizar:

- Pureza u homogeneidad en las particiones que dan lugar a los nodos hijos.
- A menor el grado de impureza, más sesgada será la distribución de clases.

¿Cómo?

- Se utilizan métricas a nivel local para determinar el mejor test de atributo.
- Estas métricas se definen con base en ciertas características descriptivas del conjunto de datos. Por ejemplo, la distribución de las clases en el nodo.

$Si\ p(i|t) = fracción\ de\ registros\ pertenecientes\ a\ la\ clase\ i\ en\ el\ nodo\ t$

$C = número\ de\ clases$

☞ Entropía: $$E(t) = -\sum_{i=1}^{C} p(i|t)\log_2 p(i|t)$$

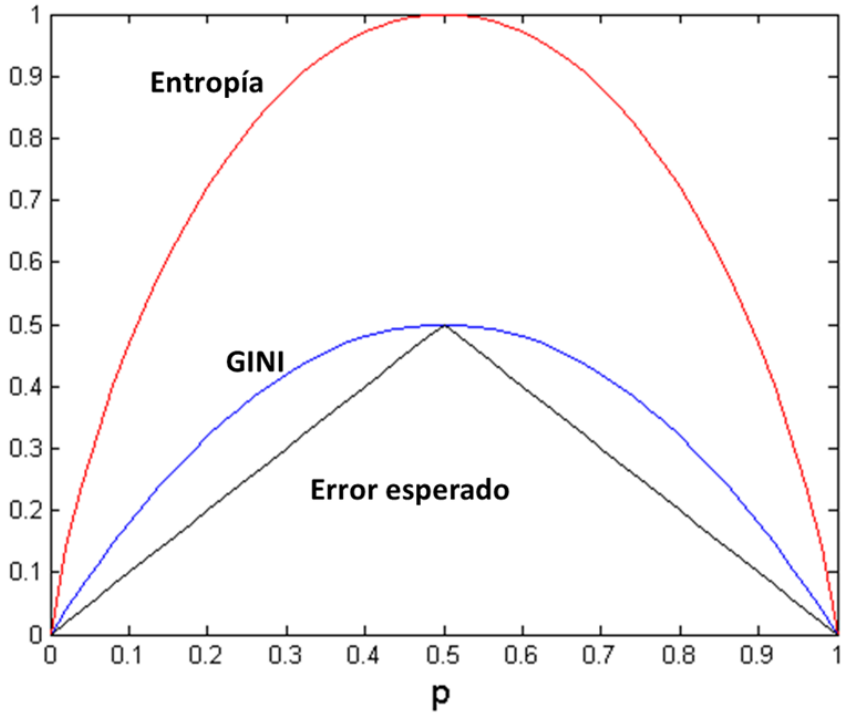
☞ Índice GINI: $$GINI(t) = 1 - \sum_{i=1}^{C} [p(i|t)]^2$$

☞ Error esperado: $$EE(t) = 1 - \max_i [p(i|t)]$$

¿Cuáles? Algunas ⇒

- Entropía
- Índice de diversidad de GINI
- Error esperado

¿Cómo usar las medidas?



Las tres medidas obtienen su valor máximo cuando la distribución de clases es uniforme

Ejemplo:

NODO	Cantidad de datos
Clase 1	0
Clase 2	6

Entropía = $- (0/6) \log_2 (0/6) - (6/6) \log_2 (6/6) = 0$

GINI = $1 - (0/6)^2 - (6/6)^2 = 0$

Error esperado = $1 - \max [(0/6), (6/6)] = 0$

NODO	Cantidad de datos
Clase 1	3
Clase 2	3

Entropía = $- (3/6) \log_2 (3/6) - (3/6) \log_2 (3/6) = 1$

GINI = $1 - (3/6)^2 - (3/6)^2 = 0.5$

Error esperado = $1 - \max [(3/6), (3/6)] = 0.5$

NODO	Cantidad de datos
Clase 1	1
Clase 2	5

Entropía = $- (1/6) \log_2 (1/6) - (5/6) \log_2 (5/6) = 0.650$

GINI = $1 - (1/6)^2 - (5/6)^2 = 0.278$

Error esperado = $1 - \max [(1/6), (5/6)] = 0.167$

- A menor el grado de impureza, más sesgada será la distribución de clases.

- Mientras más sesgada la distribución de las clases, el valor dado por las métricas será menor, alcanzando su valor mínimo cuando solo hay datos de una clase en la partición y su valor máximo cuando las clases se presentan en la misma proporción.

- Mecanismo de selección del mejor test de atributo ¿Cuál se selecciona? Aquel que induzca la partición más pura, es decir, el que proporciona el menor valor de la métrica que se esté utilizando.

- Cada algoritmo de árbol de decisión utiliza una métrica particular y es una característica que los distingue.

- Para determinar el rendimiento de un test de atributo, se compara el grado de impureza del nodo padre (*antes de la división*), con el grado de impureza de los nodos hijos (*después de la división*).

- A más grande la diferencia, mejor es el test

- La ganancia de información (Δ), es un criterio que puede utilizarse para determinar la bondad de una división:

$$\Delta info \ = \ I(NodoPadre) - \sum_{j=1}^{k} \frac{N_j}{N} I(NodoHijo_j)$$

Donde:

I = medida de impureza del nodo

N = número de registros en el nodo padre

K = número de valores del atributo (divisiones)

N_j = número de registros asociados al nodo hijo

- Entonces, primero calculamos la entropía del nodo raíz:

Instancia	Edad	Prescripción	Astigmatismo	Producción de lágrimas	Tipo
1	Joven	Miopía	No	Reducido	Ninguno
2	Joven	Miopía	No	Normal	Blandos
3	Joven	Miopía	Sí	Reducido	Ninguno
4	Joven	Miopía	Sí	Normal	Duros
5	Joven	Hipermetropía	No	Reducido	Ninguno
6	Joven	Hipermetropía	No	Normal	Blandos
7	Joven	Hipermetropía	Sí	Reducido	Ninguno
8	Joven	Hipermetropía	Sí	Normal	Duros
9	Pre-présbita	Miopía	No	Reducido	Ninguno
10	Pre-présbita	Miopía	No	Normal	Blandos
11	Pre-présbita	Miopía	Sí	Reducido	Ninguno
12	Pre-présbita	Miopía	Sí	Normal	Duros
13	Pre-présbita	Hipermetropía	No	Reducido	Ninguno
14	Pre-présbita	Hipermetropía	No	Normal	Blandos
15	Pre-présbita	Hipermetropía	Sí	Reducido	Ninguno
16	Pre-présbita	Hipermetropía	Sí	Normal	Ninguno
17	Présbita	Miopía	No	Reducido	Ninguno
18	Présbita	Miopía	No	Normal	Ninguno
19	Présbita	Miopía	Sí	Reducido	Ninguno
20	Présbita	Miopía	Sí	Normal	Duros
21	Présbita	Hipermetropía	No	Reducido	Ninguno
22	Présbita	Hipermetropía	No	Normal	Blandos
23	Présbita	Hipermetropía	Sí	Reducido	Ninguno
24	Présbita	Hipermetropía	Sí	Normal	Ninguno

$$\text{Datos} = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24\}$$

$$\downarrow$$

$$\boxed{¿?}$$ Nodo raíz

$$p(ninguno) = \frac{15}{24} = 0.625$$

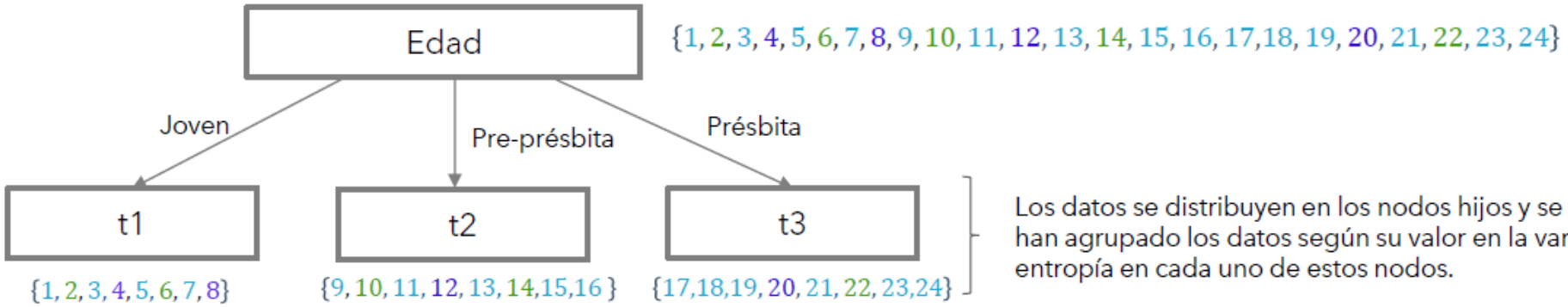
$$p(duros) = \frac{4}{24} = 0.167$$

$$p(blandos) = \frac{5}{24} = 0.208$$

$$E(nodo\ raiz) = -(0.625 \times \log_2(0.625) + 0.167 \times \log_2(0.167) + 0.208 \times \log_2(0.208))$$

$$\boxed{E(nodo\ raiz) = 0.424 + 0.431 + 0.471 = 1.326}$$

- Ahora veamos qué pasa si utilizamos la variable edad como test de atributo ¿Cuá será la ganancia de información?



Los datos se distribuyen en los nodos hijos y se genera una partición disjunta, donde se han agrupado los datos según su valor en la variable "Edad". Se calcula entonces la entropía en cada uno de estos nodos.

$$p(ninguno) = \frac{4}{8} = 0.5 \qquad p(ninguno) = \frac{5}{8} = 0.625 \qquad p(ninguno) = \frac{6}{8} = 0.75$$

$$p(duros) = \frac{2}{8} = 0.25 \qquad p(duros) = \frac{1}{8} = 0.125 \qquad p(duros) = \frac{1}{8} = 0.125$$

$$p(blandos) = \frac{2}{8} = 0.25 \qquad p(blandos) = \frac{2}{8} = 0.25 \qquad p(blandos) = \frac{1}{8} = 0.125$$

$$E(t1) = 1.5 \qquad E(t2) = 1.299 \qquad E(t3) = 1.601$$

Ahora, hay que calcular el segundo término en la ecuación de ganancia de información con base en la entropía en cada nodo hijo y el número de datos que llegan a cada uno de ellos.

$$E(edad) = \left(\frac{8}{24} \times 1.5\right) + \left(\frac{8}{24} \times 1.299\right) + \left(\frac{8}{24} \times 1.601\right) = 1.287$$

- Por último, se determina la ganancia de información:

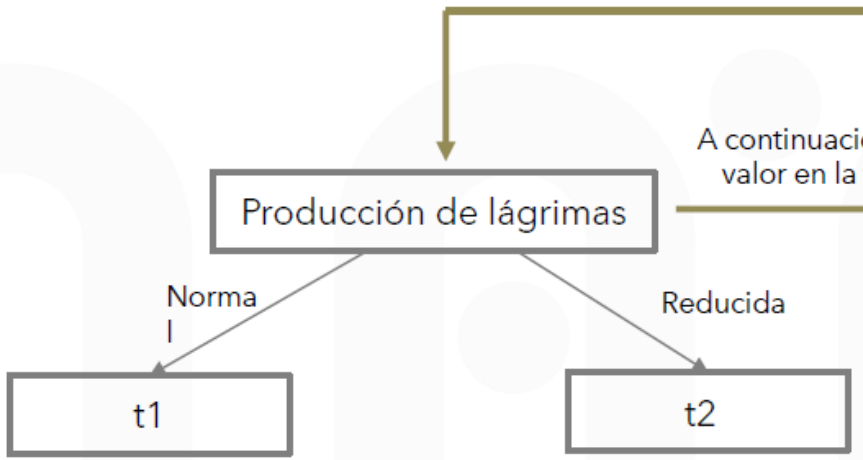
$$\Delta info = I(NodoPadre) - \sum_{j=1}^{k} \frac{N_j}{N} I(NodoHijo_j) = 1.326 - 1.287 = \boxed{0.039}$$

Entonces, si utilizamos la variable "edad" como test de atributos obtendríamos una ganancia de 0.039

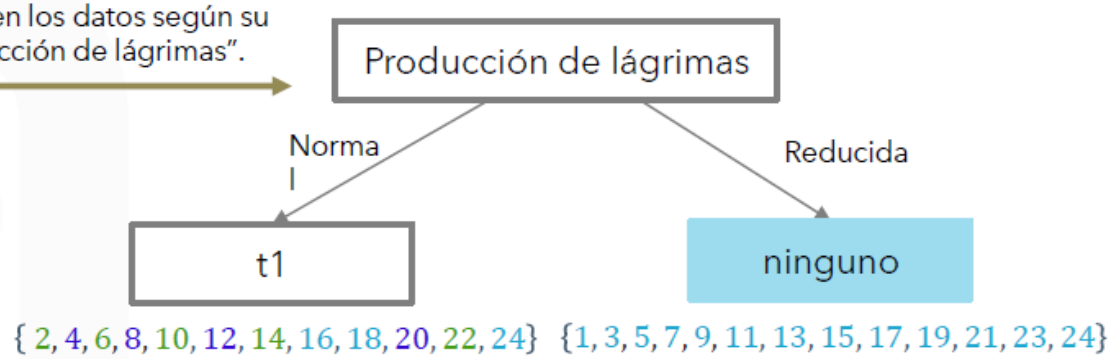
- ¿Qué pasaría si partimos de la variable “Producción de lágrimas?

Si test de atributo:	Ganancia de información
Edad	0.039
Producción de lágrimas	0.549
Astigmatismo	0.378
Prescripción	0.040

Se selecciona esta variable como test de atributo en el nodo raíz ya que proporciona la mayor ganancia de información (genera particiones más puras).



A continuación, se distribuyen los datos según su valor en la variable “Producción de lágrimas”.

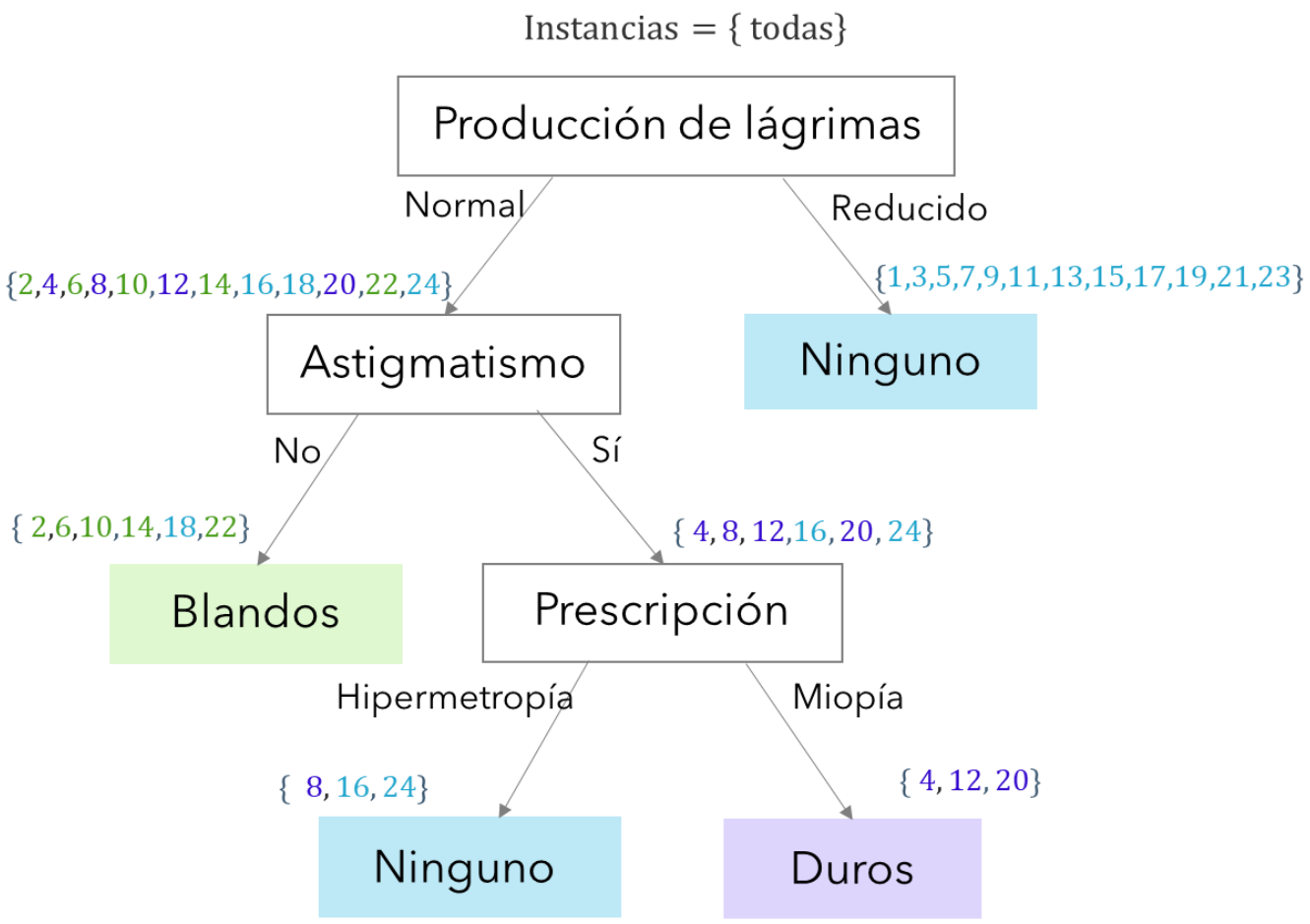


$\{2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24\}$ $\{1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 24\}$

Como la partición no es pura (hay datos de diferentes clases) se repite el proceso de buscar el mejor test de atributo, pero hora solo utilizando los datos que llegaron a este nodo

Como todos los datos de esta partición pertenecen a la misma clase (es una partición pura) este nodo se convierte en un nodo hoja con la etiqueta de clase “ninguno”.

- Una vez alcanzada la condición de parada, el árbol resultante es:



¿Condición de parada?

- Parar la expansión de un nodo cuando todos los registros pertenezcan a la misma clase.
- Parar la expansión de un nodo cuando todos los registros presenten valores similares.
- Establecer un número máximo de iteraciones.
- Establecer un número máximo de niveles o de hojas.

Además, es posible aplicar técnicas de poda para control de la complejidad. Por ejemplo, parar la expansión del árbol cuando se observa que la medida de impureza o la capacidad de predicción caen por debajo de un cierto umbral.

En general, estas técnicas son implementadas por el algoritmo.

- Una vez obtenido el árbol ¿Cómo podemos interpretarlo? Es posible trasladar el árbol a un conjunto de reglas de clasificación. Por ejemplo, para la clase “ninguno” serían:

Regla 1: Si “Producción de lágrimas” es “reducida”
entonces “lentes recomendados” es “ninguno”.

Regla 2: Si “Producción de lágrimas” es “normal”
y “Astigmatismo” es “si”
y Prescripción es “miope”
entonces “lentes recomendados” es “ninguno”.

¿Cuáles serían las reglas para las clases “Duros” y “Blandos”?

- Además, podemos deducir cuáles son las variables más importantes para hacer las predicciones: serán las que se encuentran en los primeros niveles del árbol.
- Por ejemplo, en este caso, la “Producción de lágrimas” parece ser un factor determinante en la recomendación de lentes de contacto.
- Por otra parte, vemos que algunos nodos hojas no son “puros”. Igual podemos hacer una predicción utilizando la clase mayoritaria con la que se ha etiquetado este e indicar la probabilidad de que un dato pertenezca a la clase.

❑ Algoritmo base:

Construye un árbol de decisión de manera recursiva, separando los registros (espacio de entrada), de forma sucesiva en conjuntos más puros. La pureza está determinada por la distribución de las clases en el nodo (registros que llegan a ese nodo). Entonces,

Entrada = {D: conjunto de datos = $\{(x_i, y_i)_{i = 1..N}\}$, $y_i \in \{y_1, y_2, ..y_C\}$

$D_T = D$ % (datos en el nodo T)

Comienzo_procedimiento_general

Si las instancias de D_T son de la misma clase y_k entonces T es un nodo hoja etiquetado con la clase y_k

Si D_T es un conjunto vacío entonces T es un nodo hoja etiquetado con la clase por defecto y_d.

Si D_T contiene instancias de más de una clase

Utilizar un test de atributo para dividir los datos en conjuntos más pequeños

Crear un nodo hoja por cada resultado del test

Distribuir las instancias a los nodos hijos según los resultados del test

Aplicar recursivamente procedimiento_general a cada nodo hijo

Fin-procedimiento-general

Salida = {Árbol de clasificación}

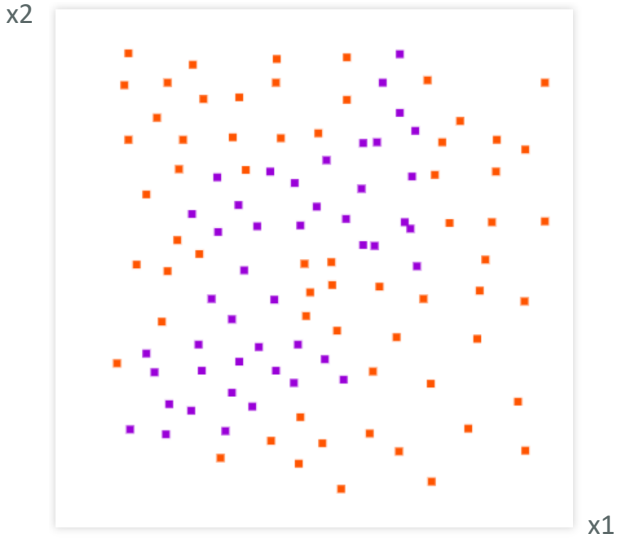
¿Condición de parada?

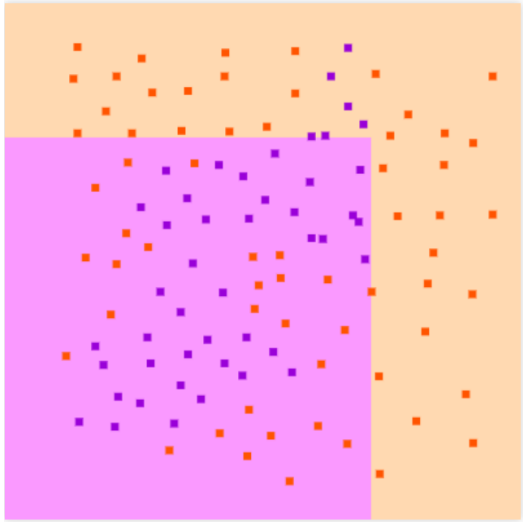
- Parar la expansión de un nodo cuando todos los registros pertenezcan a la misma clase (en el algoritmo básico).
- Parar la expansión de un nodo cuando todos los registros presenten valores similares.
- Establecer un número máximo de iteraciones.
- **Establecer un número máximo de niveles o de hojas**

Además, es posible aplicar técnicas de poda para control de la complejidad.

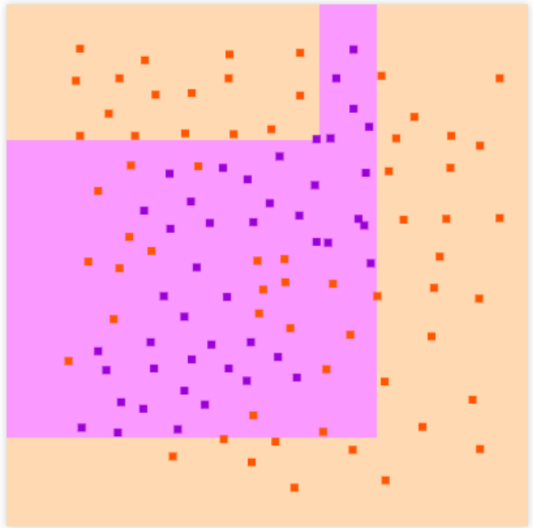
- Parar la expansión del árbol cuando se observa que la medida de impureza o la capacidad de predicción caen por debajo de un cierto umbral. También es posible establecer un número mínimo de ejemplos por nodo (pre-poda o regla de parada temprana).
- Se genera el árbol a su máximo tamaño y se aplican luego heurísticas para eliminar ramas o nodos, siempre tratando de mejorar la capacidad de predicción (post-poda).

❑ Límite de decisión y complejidad.

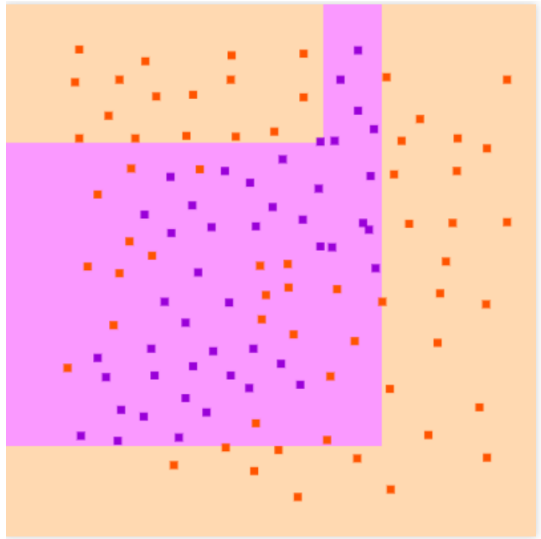




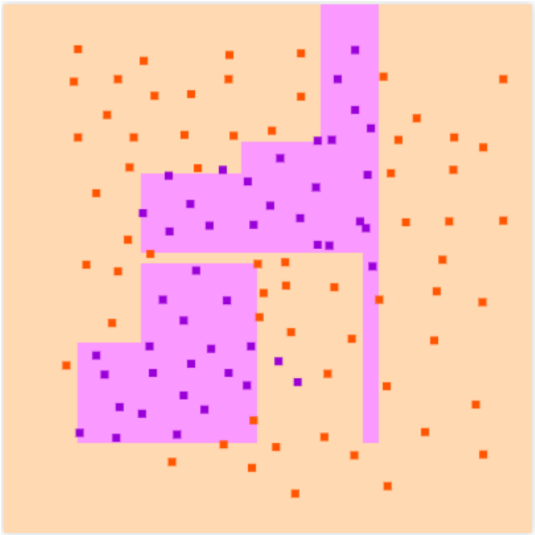
Límite de decisión un árbol de decisión (profundidad = 2)



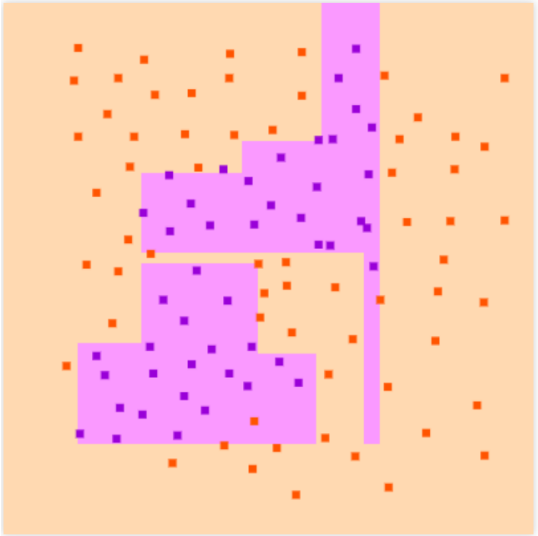
Límite de decisión un árbol de decisión (profundidad = 3)



Límite de decisión un árbol de decisión (profundidad = 5)



Límite de decisión un árbol de decisión (profundidad = 7)



Límite de decisión un árbol de decisión (profundidad = 10

https://ml-playground.com/

❑ Algunos algoritmos de árboles de decisión

ID3 (Induction Decision Trees)

- Propuesto por Quinlan, en 1983 (Referencia: Introduction to decision trees. Machine Learning. 1:81-106, 1983).
- Sólo trabaja con atributos categóricos.
- Test multivías.
- Entropía como criterio de selección, con ganancia de información.
- Criterios de parada: pureza de los nodos hojas (otros: número mínimo de datos para la división, valor de ganancia mínimo).

C4.5

- Propuesto por Quinlan, en 1993 (Referencia: C4.5. Programs for machine learning. Morgan Kaufmann)
- Test multivías. Además, puede manejar atributos numéricos y permite atributos con ausencias.
- Utiliza postpoda
- Modificación de la medida de impureza:

$$Razón\ de\ ganancia\ (gain\ ratio) = \frac{\Delta Info}{Split_Info}$$

$$Split_Info = -\sum_{i=1}^{k} p(v_i) \log_2 p(v_i)$$

$$donde\ k = número\ de\ posibles\ resultados\ del\ test$$

CART (Classification and regression tree)

- Propuesto por Breiman y otros, 1984 (Referencia: Breiman, L., Friedman, J.H., Olshen, R.A. & Stone, C.J. Classification and Regression Trees. Wadsworth, Belmont, USA).
- Sólo test binarios.
- Puede manejar atributos numéricos.
- Utiliza postpoda.
- Índice de diversidad de GINI como criterio de selección, con ganancia de información.

- OTROS: CHAID, REPTree, randomTree, BOAT, …

¿Cómo funciona el algoritmo a utilizar?

¿Qué requerimientos tiene?

¿Cómo ajustar sus parámetros?

	Atributo 1	Atributo 2		Atributo d	Clase
Instancia 1					
Instancia 2					
Instancia n					

Dependiendo del algoritmo, solo atributos categóricos. Algunos también pueden trabajar con atributos numéricos

Aprendizaje supervisado
Tarea de clasificación

Árboles de decisión

Procedimiento de optimización local (basado en búsqueda heurística)

Criterio a optimizar: GINI, entropía

Hiperparámetros:

Profundidad del árbol, número mínimo de datos para la división, ….

Algoritmo base (Hunt)

Entrada = {D: conjunto de datos = $\{(x_i, y_i)_{i=1..N}\}$, $y_i \in \{y_1, y_2, ..y_C\}$

$D_T = D$ % (datos en el nodo T)

Comienzo_procedimiento_general

Si las instancias de D_T son de la misma clase y_k entonces T es un nodo hoja etiquetado con la clase y_k

Si D_T es un conjunto vacío entonces T es un nodo hoja etiquetado con la clase por defecto y_d.

Si D_T contiene instancias de más de una clase

Utilizar un test de atributo para dividir los datos en conjuntos más pequeños

Crear un nodo hijo por cada resultado del test

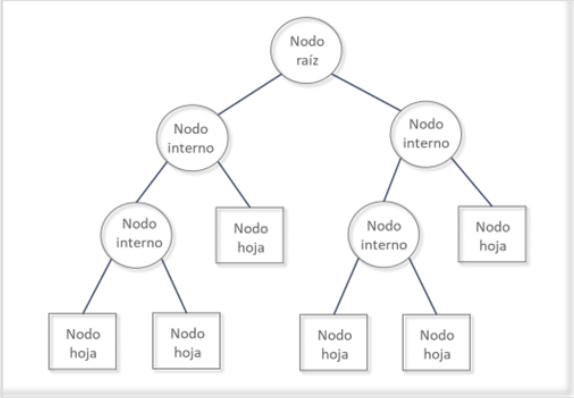
Distribuir las instancias a los nodos hijos según los resultados del test

Aplicar recursivamente procedimiento_general a cada nodo hijo

Fin-procedimiento-general

Salida = {Árbol de clasificación}

Modelo estimado

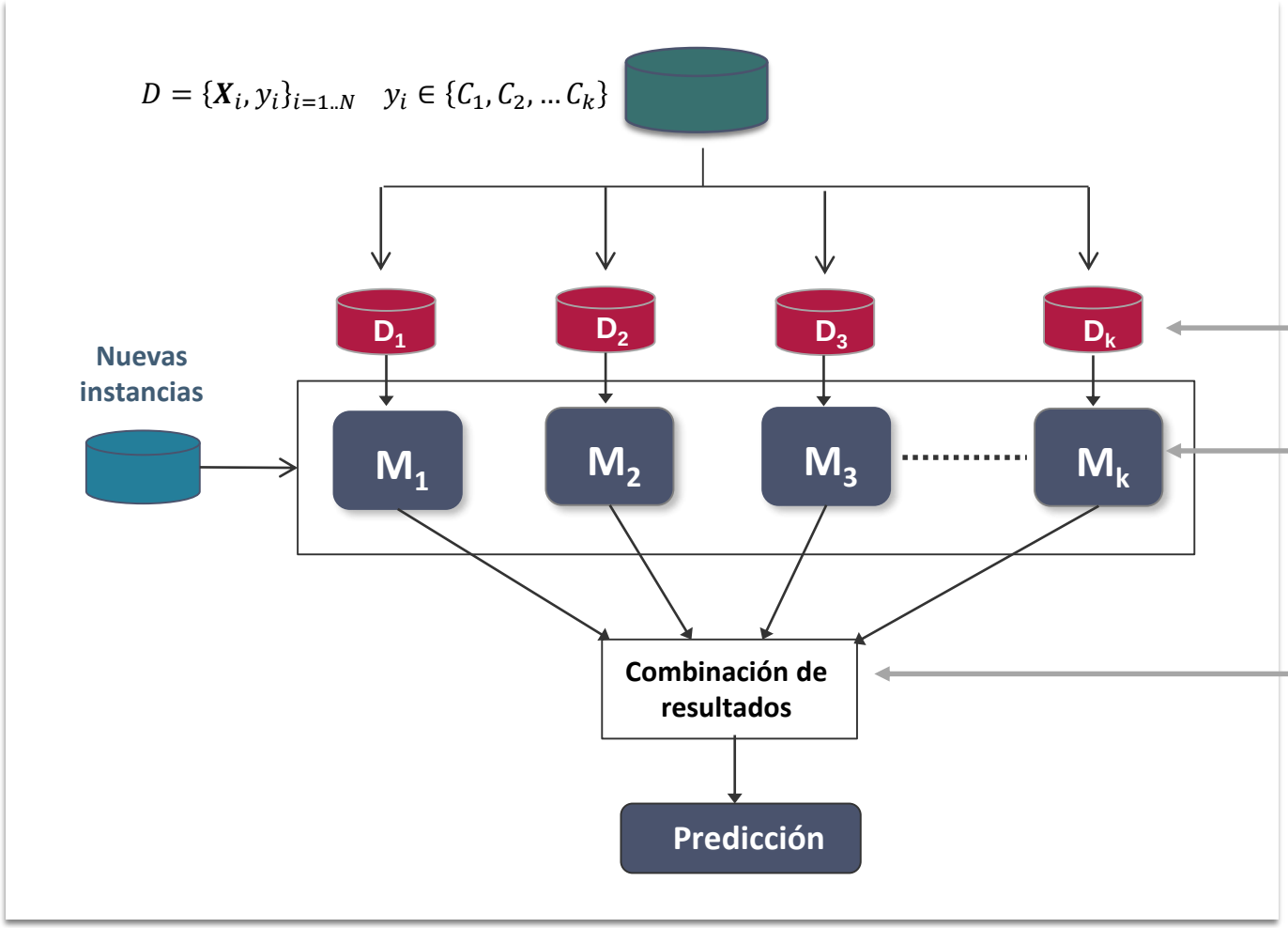


❑ ¿Cómo utilizamos los algoritmos basados en árboles?

Tarea	Problema de aprendizaje	Parámetros del modelo	Hiperparámetros	Características y contextos de aplicación
Clasificación (también hay algoritmos para aprender árboles de regresión)	Procedimiento de búsqueda heurística del mejor test de atributo, con base en una métrica que mide la pureza de una partición.	Test de atributos La selección del mejor test se realiza utilizando un criterio de pureza, por ejemplo Entropía, índice GINI, entre otros.	Profundidad del árbol. Número de Hojas. Número mínimo de instancias para hacer las divisiones (seleccionar el test de nodo),… *Algunas implementaciones suministran control de la complejidad a través de pruning.*	Algunos algoritmos son robustos a las ausencias. Trabajan bien con mezclas de atributos (numéricos y categóricos). No requieren escalado de las variables numéricas (normalización o estandarización). La presencia de atributos redundantes no afecta la construcción del árbol; sin embargo, puede suceder que se seleccionen atributos irrelevantes. Tienden a tener una alta varianza (ante pequeñas variaciones de los datos pueden generar árboles muy diferentes). Pueden ser interpretados fácilmente (comprensibles para usuarios no expertos). Al hacer una selección de atributos, pueden identificarse aquellos que son más importantes para la tarea (sumario de la importancia de las variables). Utilizados en aquellas aplicaciones en las que se requiere un modelo interpretable. Puede ser usado para ingeniería de características.

Combinación de modelos (*ensembles*)

Modelo compuesto:



Aspectos de diseño:

¿Cómo se construyen los conjuntos de datos D_i, i=1..k?

¿Cuáles algoritmos se utilizan para la construcción de los diferentes modelos, M_j, j=1…k?

¿Cuál es la regla de decisión que se utiliza para combinar las predicciones de los diferentes clasificadores y producir una única salida?

❑ Bagging

☞ Construcción de los conjuntos de datos D_i, i=1..k.

Para la construcción de cada conjunto de datos se realiza un muestreo con reemplazo de N datos a partir del conjunto de aprendizaje D. Este muestreo crea diversidad entre los diferentes modelos.

☞ Algoritmos a utilizar para el aprendizaje de los diferentes modelos, M_j, j=1…k.

Cada modelo puede ser construido con una máquina de aprendizaje diferente.

☞ Regla para combinar las predicciones de los diferentes clasificadores y producir una única salida.

Se utiliza un esquema de votación simple. Se selecciona la clase que tenga más votos.

Puede aplicarse para la predicción de valores numéricos tomando el promedio sobre cada predicción dada por un modelo.

Ejemplo →

☞ Construcción de los conjuntos de datos D_i, i=1..k.

Para la construcción de cada conjunto de datos se realiza un muestreo con reemplazo de N datos a partir del conjunto de aprendizaje D. Pero también se realiza una selección aleatoria de atributos (muestreo de subespacios). Estos atributos serán los candidatos a test de división en cada nodo del árbol.

☞ Algoritmos a utilizar para el aprendizaje de los diferentes modelos, M_j, j=1…k.

Cada modelo se construye con un algoritmo de árbol de decisión; este algoritmo es el mismo para todos.

☞ Regla para combinar las predicciones de los diferentes clasificadores y producir una única salida.

Se utiliza un esquema de votación simple. Se selecciona la clase que tenga más votos

Random Forest

❑ Boosting

☞ Construcción de los conjuntos de datos D_i, i=1..k.

Se realiza un muestreo con reemplazo y se asignan pesos a cada instancia del conjunto de datos. Los clasificadores se entrenan iterativamente.
Después de entrenar un clasificador Mj, los pesos de las instancias se actualizan para que el próximo clasificador a entrenar Mi+1 "ponga mas atención" a las instancias que no fueron correctamente clasificadas por Mi.

☞ Algoritmos a utilizar para el aprendizaje de los diferentes modelos, M_j, j=1…k.

Cada modelo puede ser construido con una máquina de aprendizaje diferente, conocidos como aprendices débiles.

☞ Regla para combinar las predicciones de los diferentes clasificadores y producir una única salida.

Se utiliza un esquema de votación simple y se selecciona la clase que tenga más votos. Sin embargo, cada voto tendrá un peso que depende de la exactitud del clasificador.

Ejemplo

☞ Construcción de los conjuntos de datos D_i, i=1..k.

Inicialmente se asigna un peso $1/n$ a cada instancia del conjunto de datos (n = número de datos).
En la iteración i se construye el conjunto de entrenamiento D_i utilizando muestreo con reemplazo, en el cual la probabilidad de que una instancia sea seleccionada depende de su peso.

☞ Algoritmos a utilizar para el aprendizaje de los diferentes modelos, M_j, j=1…k.

Puede utilizarse cualquier algoritmo. Al entrenar un clasificador, los pesos de las instancias se actualizan dependiendo de cómo fueron clasificadas por este: si hay error se incrementa su peso, si no hay error este se disminuye.

☞ Regla para combinar las predicciones de los diferentes clasificadores y producir una única salida.

El voto de cada clasificador se calcula con base en la cantidad de error que comete al clasificar los datos.

Adaboost

Otros:

- Gradient Tree Boosting
- Histogram-Based Gradient Boosting
- Voting Classifier
- Voting Regressor
- Stacked generalization. …

Al igual que AdaBoost, Gradient Boosting agrega predictores de manera secuencial, para corregir a su predecesor.

Pero en lugar de ajustar los pesos de las instancias en cada iteración, este método intenta ajustar los errores residuales cometidos por el predictor anterior, utilizando descenso por el gradiente.

Se puede utilizar cualquier estimador

Por ejemplo, en scikit-learn:

```
from sklearn.ensemble import BaggingClassifier
from sklearn.neighbors import KNeighborsClassifier
bagging = BaggingClassifier(KNeighborsClassifier(),
                            max_samples=0.5, max_features=0.5)
```

max_samples indica cuántos datos, del conjunto de entrenamiento serán utilizados por cada clasificador; max_features cuántas características.

Por ejemplo, otra librería:

- XGBoost (https://xgboost.readthedocs.io/en/stable/)

¿Cuándo el modelo es más complejo?

Próxima sesión

- Preparación de textos
- Modelos generativos.
- Naïve Bayes
- Aprendizaje basado en instancias.